

Telegram-to-Nostr Bridge

Production Startup & Configuration Guide

Overview: This automation script monitors public Telegram channels via web scraping, processes embedded images by uploading them to a Blossom media server, and broadcasts the posts out to your chosen Nostr relays.

1. System Prerequisites

Before deploying the bot, make sure your target system contains the standard modern runtime environment:

- **Node.js** (v18 or higher recommended)
- **npm** (bundled with Node.js package manager)

2. Project Installation

Set up your local directory and retrieve the required dependencies:

```
mkdir telegram-nostr-bot
cd telegram-nostr-bot
# Save your main script as bot.js inside this folder
npm init -y
npm install axios cheerio nostr-tools
```

3. Configuration Blueprint (`config.json`)

Create a `config.json` file in your root workspace directory. This isolates your private keys and target parameters securely.

```
{
  "private_key_hex": "YOUR_NOSTR_PRIVATE_KEY_IN_HEX",
  "channels": [
    "TelegramChannel1",
    "TelegramChannel2"
  ],
  "blossom_server_url": "https://blossom.primal.net",
  "relay_urls": [
    "wss://relay.damus.io",
    "wss://nos.lol",
    "wss://relay.snort.social",
    "wss://relay.primal.net"
  ]
}
```

Configuration Parameter Breakdown

- **private_key_hex** : Your Nostr account private key strictly formatted in *hexadecimal format* (avoid entering an ``nsec`` string).
- **channels** : Array of public target Telegram usernames (exclude the `https://t.me/s/` URL prefix).
- **blossom_server_url** : Blossom-compatible endpoint used to securely host image payloads prior to Nostr broadcast.
- **relay_urls** : List of target Nostr communication relay nodes.

4. Launching the Bot

Option A: Manual Verification Mode

Execute the script directly to verify setup logs and test connectivity:

```
node bot.js
```

Option B: Production Background Service (PM2)

For persistent execution, use a process manager like PM2 to auto-restart the bot on system reboots or unhandled crashes:

```
npm install -g pm2
pm2 start bot.js --name "telegram-nostr-bot"
pm2 startup
pm2 save
```

5. Architecture & Troubleshooting

- **Polling Schedule**: Evaluates public Telegram feeds immediately upon launch, then repeats execution every **5 minutes**.
- **State Deduplication**: Successfully dispatched post IDs are automatically tracked and mirrored inside `sent_posts.json` to prevent duplicate broadcast loops.
- **Config Error Fix**: If you spot an `Error reading config.json` notice, confirm the JSON syntax is valid and placed in the exact directory as `bot.js`.
- **Channel Constraints**: Target channels must remain public and accessible via standard web preview previews (``t.me/s/...``). Private feeds cannot be read using widget scraping.